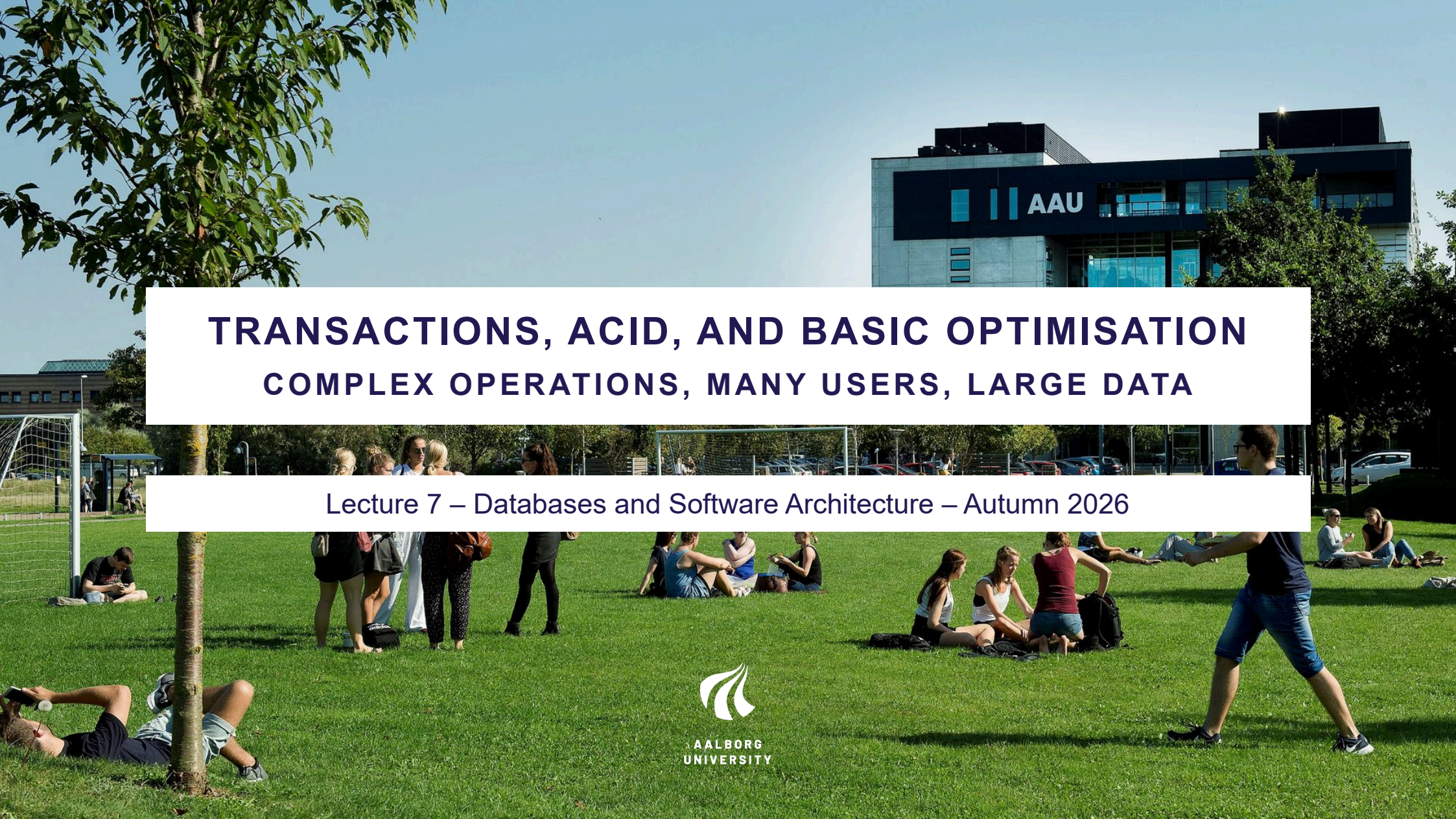


TRANSACTIONS, ACID, AND BASIC OPTIMISATION

COMPLEX OPERATIONS, MANY USERS, LARGE DATA

Lecture 7 – Databases and Software Architecture – Autumn 2026

AALBORG UNIVERSITY

BEFORE WE START: SOME CHANGES TO THIS COURSE

WHY WE ARE CHANGING THINGS: AI

1. **AI is changing the world** – how we live, how we learn, and how we work.
2. **So some ways of teaching are changing.** Lectures 1–5 taught what a database course has always taught: how to **code** – `CREATE TABLE`, `SELECT`, `JOIN`.
3. **But coding was never the goal.** It is a **means** – a way to reach understanding.

What should you understand and what is AI taking over?

ON THE SURFACE, AI IS EATING EVERYTHING

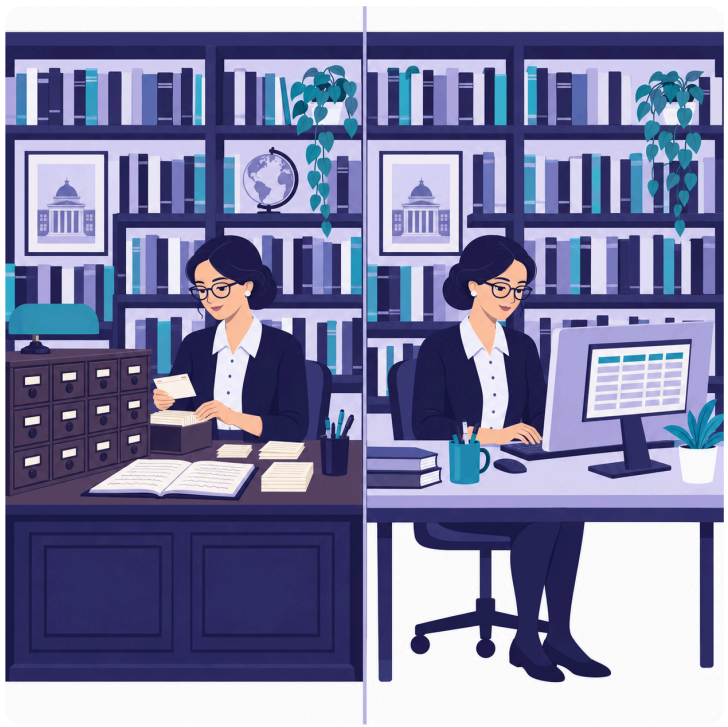


Look closer. What AI eats is the **surface** of every task:

- the **execution** – typing it out
- the **repetitive** part – the boilerplate, the tenth `INSERT`
- everything that **can be written down completely** – once you can say exactly what you want, a machine can do it

What it hands back to you is the core – heavier than before, because now it is **all** that is left: knowing what to want, and knowing whether you got it.

EXAMPLE 1: THE LIBRARIAN, BEFORE AND AFTER



CORE SKILL

Understanding the library

its entities (books, copies, members, loans), how they relate, and what every operation means: lend, return, reserve, withdraw

BEFORE

Records kept by hand

card catalogues, ledgers, index cards

AFTER

Records kept in a database

tables, keys, queries

The information revolution changed the tool – not the understanding of the library.

EXAMPLE 2: THE DATABASE ENGINEER, BEFORE AND AFTER AI



CORE SKILL

Understanding the data

its entities and relationships, every kind of operation on it, and the experience to keep it safe and stable

BEFORE

Writing SQL and code by hand

every table, query and transaction typed out

AFTER

Specifying and judging

say precisely what is needed; check what an AI writes

AI is changing how the code gets written – not the understanding of the data.

SO THIS COURSE SEPARATES THE TWO

From here on, we keep two things apart, so you always know which one you are looking at.

WHAT LASTS

Problems, understanding and judgment

What a database exists to solve, and how to decide well – the part to keep in your head.

WHAT CHANGES

The tool of the day

The SQL, the syntax, the code – which an AI can increasingly write for you, once you can say what you need.

To make the separation visible, I have built a **mind map** of the course.

ONE MIND MAP, THREE PERSPECTIVES

Perspective	What it holds
1 · Key points	Understanding of data: the real problems and scenarios that come with it, and how today's databases deal with such data in general. The tools may change; these problems will not
2 · Example	One running example, the library, used throughout the course. It is the bridge that connects the key points and the database practice to a real-world scenario, so both are easier to understand
3 · Practice	The concrete practice: how to create the tables, how to write the SQL

Every slide from here on carries one of these three labels in its top-right corner.

THE SHAPE OF THE COURSE

Lectures	Content
1–8	Databases – this mind map covers Lectures 1–8
9–11	Software architecture – teaching materials released later this month
12	Questions and answers

Now, let us open the mind map of first part ‘Databases’ and walk through it.

BACK TO LECTURE 7: KEEPING DATA RIGHT WHEN THINGS GET COMPLEX

Lecture 6 changed data one statement at a time.
Today, we need to consider more complex scenarios: more complex operations, many people collaborating, and large data

WHERE WE ARE

Lecture 6 changed data **one statement at a time**. Real work is more complex. Today's core question:

How does a database keep data stable, accurate and efficient – in complex situations?

NEED 1 **A complex operation – try it safely, then make it final.**

Several steps, tried and checked before they count. What if one goes wrong?

NEED 2 **Many people work on the same data at once.**

Two desks reach for the last copy; the overdue clean-up runs while the desks keep working.

NEED 3 **Large data – and still fast.**

The loan log reaches 500,000 rows. How do we still find one member's loans fast?

RECAP: THE LIBRARY SCHEMA

The same seven tables as every lecture since Lecture 3, plus one big table for need 3:

- `Copy(book_id, copy_number, status)` — `status` is `'available'` or `'on loan'`
- `Member(member_id, first_name, last_name, email, join_date, referred_by)`
- `Loan(loan_id, member_id, book_id, copy_number, loan_date, due_date, return_date)`
- `Book`, `Author`, `BookAuthor`, `BookGenre` — unchanged
- **New:** `LoanLog` — a synthetic event log with 500,000 rows, only for the indexing demos (`extras/sql/05_bulk_loan_log.sql`)

NEED 1: A COMPLEX OPERATION, DONE SAFELY

A complex change is worked out step by step – tried, checked, and only then made final.

SQL: `BEGIN`, `COMMIT`, `ROLLBACK`, `SAVEPOINT`

NEED 1: A COMPLEX OPERATION, DONE SAFELY

A complex change takes several steps – even borrowing a book is two. The database helps in three ways.

WHAT THE DATABASE CAN PROVIDE FOR YOU

F1.1 All or nothing

Both writes happen, or neither does.

F1.2 Go back to a save point

Try a step, check it, go back if it is wrong – as often as needed.

F1.3 Stable once committed

After `COMMIT` returns, even a crash cannot undo it.

WHAT YOU MUST DECIDE

Design one complete operation as one transaction

What goes between `BEGIN` and `COMMIT` is yours to decide.

- **Inside:** every step – both writes of a loan.
- **Try** a step after a save point; **check** the result.
- **Go back** if it is wrong, and try again.
- `COMMIT` only when every check passes.

WORKING OUT A COMPLEX TASK: LIKE A VIDEO GAME

A complex task is rarely right the first time. A transaction works like one game level:

In a game level	In a complex task	In SQL
Start the level	Start – nothing is final yet	`BEGIN;`
Save inside the level	Mark a point to come back to	`SAVEPOINT s;`
Try – and lose	Try a step: the check says wrong	`UPDATE ...;`
Load the save	Go back to the point	`ROLLBACK TO SAVEPOINT s;`
Try again until you win	Try again: the check says right	`UPDATE ...;`
Clear the level	Make it final; no more going back	`COMMIT;`

Save points live inside one level. After `COMMIT` the next transaction is the next level – you can never load a save from an earlier one.

WHAT THE DATABASE OFFERS FOR NEED 1

	Function	SQL	What it does
F1.1	All or nothing	`BEGIN ... COMMIT` / `ROLLBACK`	Everything between them counts as one unit – a *transaction*. `ROLLBACK` undoes all of it
F1.2	Go back to a save point	`SAVEPOINT`, `ROLLBACK TO SAVEPOINT`	Returns to a named point in the transaction; what came before it stays
F1.3	Stable once committed	`COMMIT`	Returns only after the change is safely on disk – a crash afterwards cannot undo it

Without `BEGIN`, every statement is its own one-statement transaction (*autocommit*). The safety net from Lecture 6 was a transaction all along.

What it cannot do for you: design one complete operation as one transaction – what goes inside, and when to `COMMIT`.

THE PROBLEM: TWO WRITES, ONE CRASH

```sql
UPDATE Copy SET status = 'on loan' WHERE book_id = 5 AND copy_number = 1;
-- <- the program stops here: an error, a crash
INSERT INTO Loan (member_id, book_id, copy_number, due_date)
VALUES (12, 5, 1, CURRENT_DATE + 14);
```

Read it as: *borrow copy (5, 1) for member 12 – as two separate statements.*

- **Each statement is final on its own.** A crash after the first leaves the copy “on loan”, with no loan that says who has it or when it is due.
- **The textbook version is a bank transfer:** take 100 from Alice, then give 100 to Bob. A crash in between, and the 100 is simply gone.

BEGIN, COMMIT AND ROLLBACK

Keep both writes

```
BEGIN;
UPDATE Copy SET status = 'on loan'
WHERE book_id = 5 AND copy_number = 1;
INSERT INTO Loan (member_id, book_id,
  copy_number, due_date)
VALUES (12, 5, 1, CURRENT_DATE + 14);
COMMIT;
```

Throw both away

```
BEGIN;
UPDATE Copy SET status = 'on loan'
WHERE book_id = 5 AND copy_number = 1;
-- something goes wrong:
-- the loan should not happen
ROLLBACK;
```

Read it as: *make the two writes one unit – then keep it, or throw it all away.*

- `BEGIN` **opens the unit.** Every statement after it belongs to the same transaction.
- `COMMIT` **keeps both writes, together.** A crash before it, and neither survives – never "just the first".
- `ROLLBACK` **throws both away.** It is what undid the 157-row `UPDATE` in Lecture 6.

SAVEPOINT: GO BACK TO A SAVE POINT

```
BEGIN;                                      -- closed two weeks: 27 loans fall due
SAVEPOINT try;
UPDATE Loan SET due_date = due_date + 14
WHERE return_date IS NULL;                  -- UPDATE 63: 36 too many
ROLLBACK TO SAVEPOINT try;                  -- wrong: go back, try again
UPDATE Loan SET due_date = due_date + 14
WHERE return_date IS NULL
  AND due_date BETWEEN CURRENT_DATE AND CURRENT_DATE + 13;    -- UPDATE 27
COMMIT;                                     -- right: now make it final
```

- **Trial and error, safely.** Nothing is final until `COMMIT`; go back to the save point as often as you need. Verified on a fresh load: 63 loans on the first try, exactly 27 on the second.
- **Only inside this transaction.** A save point disappears at `COMMIT` or `ROLLBACK` – it is a checkpoint, not a version history.

COMMIT MEANS CONFIRMED – DATA IS SAVED ON DISK

```
BEGIN;
UPDATE Copy SET status = 'on loan' WHERE book_id = 5 AND copy_number = 1;
INSERT INTO Loan (member_id, book_id, copy_number, due_date)
VALUES (12, 5, 1, CURRENT_DATE + 14);
COMMIT;   -- returns only after the change is written to the log on disk
```

Read it as: *“done” means COMMIT returned – not that the INSERT ran.*

- **The write-ahead log.** Every change is written to a log on disk **before** COMMIT returns. After a crash, PostgreSQL replays the log.
- **Before COMMIT returns, nothing is promised; after it, everything is.**
- How the log works inside is *Database Systems*, 6th semester. Today, the promise is what matters.

INDEX: NEED 1

Function	What the database provides	Read more
F1.1 All or nothing	`BEGIN` / `COMMIT` / `ROLLBACK` `BEGIN; UPDATE Copy ...; INSERT INTO Loan ...; COMMIT;`	DBS 4.3, 17.1
	the bank-transfer example two `UPDATE`s on `Account` – both, or neither	DBS 17.1
F1.2 Go back to a save point	`SAVEPOINT` `SAVEPOINT try; ... ROLLBACK TO SAVEPOINT try;`	PG `SAVEPOINT`
F1.3 Stable once committed	durability; the write-ahead log `COMMIT` returns only after the log is on disk	DBS 17.1, 19.1

DBS = *Database System Concepts*, Silberschatz, Korth & Sudarshan, 7th ed. (17.1 = chapter 17, section 1); PG = the PostgreSQL documentation, page for that command.

NEED 2: MANY PEOPLE WORK AT ONCE

One `BEGIN ... COMMIT` is one transaction – and now many are open at the same time.

SQL: isolation levels, row locks, deadlock detection

NEED 2 BACKGROUND: MANY TRANSACTIONS AT ONCE

Need 1 made one task safe. Now many unfinished tasks run side by side – two questions.

1 · WHAT DOES EACH ONE SEE – AND WHAT MUST STILL HOLD WHEN IT WRITES? – THREE NEEDS

Real-time

I always see the latest commits.

Lending at the desk: you want the latest information at the moment you act.

No overwrite

I see one snapshot; what I write has not been written by others since.

Overdue clean-up: list and count, then mark the copies lost.

Valid decision

I see one snapshot; what my decision rests on has not been changed by others since.

Loan limit: count a member’s loans, then lend – the count must still hold.

2 · WHAT IF TWO CHANGE THE SAME DATA? – TWO PROBLEMS

Write conflicts

Two desks lend the same copy at the same moment – whose change counts?

Deadlocks

Each desk holds a copy the other one needs – do both wait forever?

NEED 2: SEE, THEN WRITE – THREE ISOLATION LEVELS

Three needs – three isolation levels. You pick one for every transaction.

WHAT THE DATABASE CAN PROVIDE FOR YOU

REAL-TIME

F2.1.1 Latest commits

Each statement sees the latest commits. Never refuses.

`READ COMMITTED` (default) · *The desk, lending a copy*

NO OVERWRITE

F2.1.2 Snapshot + write protection

One moment for the whole transaction. Row changed since? Refused.

`REPEATABLE READ` · *The overdue clean-up*

VALID DECISION

F2.1.3 Snapshot + basis protection

Also guards your decision's basis: one is refused.

`SERIALIZABLE` · *The loan limit: count, then lend*

WHAT YOU MUST DECIDE

Choose the isolation level

For each transaction:

- Real-time, no overwrite, or valid decision?
- Too weak: wrong numbers.
- Too strong: needless retries.
- `SERIALIZABLE`: all involved must use it.

NEED 2: WHO WAITS

Two problems the database handles by itself – you only need to know them.

WHAT THE DATABASE CAN PROVIDE FOR YOU

WRITE CONFLICTS

F2.2 Row waits

A second writer of the same row waits for the first one's `COMMIT`.

Two desks, one copy: desk B waits, then gets `UPDATE 0`

DEADLOCKS

F2.3 Deadlock detection

Two waiting for each other: after about a second, the database cancels one and undoes its work.

Anna's and Ben's desks: one is cancelled, the other commits

WHAT YOU MUST DECIDE

Talk to your collaborators

Split the work so nobody waits long:

- Agree on one order for locking rows.
- Keep transactions short.
- Put the check inside the `UPDATE`.
- Retry what was refused or cancelled.

IN THE LIBRARY: SEE, THEN WRITE

The three needs at the desks.

REAL-TIME

The desk, lending a copy

Desk A marks copy (5, 1) *on loan*, then takes it back (`ROLLBACK`). Desk B must never see that half-done loan – but once a loan is committed, its next look must show it.

NO OVERWRITE

The overdue clean-up

It lists the copies more than 30 days overdue – 11 – and marks them *lost*. Meanwhile a desk takes back copy (47, 2). List and count must be from one moment – and (47, 2) must not be marked *lost*.

VALID DECISION

The loan limit

At most four open loans. Member 2 has three. Two desks count at the same moment, both see 3, both lend – now member 2 has 5.

IN THE LIBRARY: WHO WAITS

The two problems at the desks – when two change the same rows.

WRITE CONFLICTS

The same copy

Desk A starts lending copy (1, 1). Half a second later, desk B tries the same copy. Does B say “done”, “not available” – or wait?

DEADLOCKS

Waiting for each other

Anna and Ben both want copies (1, 1) and (3, 1). Desk A takes (1, 1) first; desk B takes (3, 1) first. Now each desk waits for the copy the other one holds.

A trap that is yours to avoid: ask “is it available?” with a `SELECT`, then lend with a plain `UPDATE` – both desks see *yes*, both lend. Two receipts for one book.

1 · KEY POINT

WHAT THE DATABASE OFFERS FOR NEED 2

	Function	SQL	What it does
F2.1.1	Latest commits	`READ COMMITTED` – the default	Each statement sees what was committed when it started; never unfinished work; never refuses
F2.1.2	Snapshot + write protection	`REPEATABLE READ`	One snapshot from the first statement; changing a row others committed a change to since: refused
F2.1.3	Snapshot + basis protection	`SERIALIZABLE`	Also tracks what each read, committed or not; refuses one if no one-at-a-time order fits – if all use it
F2.2	Row waits	row lock – automatic	A changed row stays locked until `COMMIT`; a second writer waits
F2.3	Deadlock detection	automatic	After about a second, one of two waiting transactions is cancelled

What it cannot do for you: choose the isolation level or the lock order, put the check inside the `UPDATE` – or retry what it refused.

READ COMMITTED: REAL-TIME, THE DEFAULT

```
B: BEGIN;
B: SELECT status FROM Copy WHERE book_id = 5 AND copy_number = 1;   -> available
A: UPDATE Copy SET status = 'on loan' WHERE book_id = 5 AND copy_number = 1;   (commits)
B: SELECT status FROM Copy WHERE book_id = 5 AND copy_number = 1;   -> on loan
B: COMMIT;
```

Read it as: *the same SELECT, twice, in one transaction – two answers.*

- **No dirty reads.** While A had not committed, B saw *available*. Verified – even when B asks for READ UNCOMMITTED, which PostgreSQL treats exactly like READ COMMITTED.
- **But non-repeatable reads happen.** Each statement takes a fresh snapshot, so the second read sees A's committed change.

REPEATABLE READ: ONE SNAPSHOT

```
B: BEGIN ISOLATION LEVEL REPEATABLE READ;
B: SELECT status FROM Copy WHERE book_id = 5 AND copy_number = 1;  -> available  (read 1)
A: UPDATE Copy SET status = 'on loan' WHERE book_id = 5 AND copy_number = 1;  (commits)
B: SELECT status FROM Copy WHERE book_id = 5 AND copy_number = 1;  -> available  (read 2, the same!)
B: COMMIT;
B: SELECT status FROM Copy WHERE book_id = 5 AND copy_number = 1;  -> on loan  (read 3, a new one)
```

Read it as: *one snapshot for the whole transaction – whatever commits elsewhere meanwhile.*

- **No non-repeatable reads.** Read 2 still sees the snapshot taken at read 1.
- **No phantom reads either, in PostgreSQL.** Verified: counting member 12′s open loans gave 1, then 2 under `READ COMMITTED` – but the same count twice under `REPEATABLE READ` stayed the same.

REPEATABLE READ: NO OVERWRITE

```
B: BEGIN ISOLATION LEVEL REPEATABLE READ;                    -- the overdue clean-up
B: CREATE TEMP TABLE todo AS SELECT book_id, copy_number FROM Loan
     WHERE return_date IS NULL AND due_date < CURRENT_DATE - 30;    -- 11 copies
A: the desk takes back copy (47, 2): Loan returned, Copy 'available'; COMMIT;
B: SELECT count(*) FROM Loan WHERE return_date IS NULL AND due_date < CURRENT_DATE - 30;   -> 11
B: UPDATE Copy SET status = 'lost' WHERE (book_id, copy_number) IN (SELECT * FROM todo);
ERROR:  could not serialize access due to concurrent update
```

Read it as: *the clean-up wants to mark lost a copy that came back after its snapshot was taken.*

- **Verified** on a fresh load. `READ COMMITTED`: the count drops to 10, yet `UPDATE 11` marks (47, 2) *lost* although it is back on the shelf. `REPEATABLE READ`: the count stays 11 – and the `UPDATE` is refused.
- **Your program must retry.** The new snapshot no longer lists (47, 2).

SERIALIZABLE: A VALID DECISION

```
-- the rule: at most 4 open loans. Member 2 has 3. Both desks use SERIALIZABLE.
A: BEGIN ISOLATION LEVEL SERIALIZABLE;
A: SELECT count(*) FROM Loan WHERE member_id = 2 AND return_date IS NULL;   -> 3
B: BEGIN ISOLATION LEVEL SERIALIZABLE;
B: SELECT count(*) FROM Loan WHERE member_id = 2 AND return_date IS NULL;   -> 3
A: INSERT INTO Loan (member_id, book_id, copy_number, due_date) VALUES (2, 4, 1, ...);  COMMIT;
B: INSERT INTO Loan (member_id, book_id, copy_number, due_date) VALUES (2, 5, 1, ...);
ERROR:  could not serialize access due to read/write dependencies among transactions
```

Read it as: *each desk counted before the other lent – no one-at-a-time order explains both.*

- **Verified** on a fresh load: under `READ COMMITTED` and `REPEATABLE READ` both desks lend – member 2 ends with 5. Under `SERIALIZABLE`, desk B is refused – 4.
- **No row is shared,** so no lock helps. The database tracks what each transaction read; the rule itself stays in your program.
- **Only if every desk uses `SERIALIZABLE`** – and refused work is retried.

WHICH LEVEL PREVENTS WHAT

1 · KEY POINT

F2.1.1 -- F2.1.3 ↑

Level	Dirty read	Non-repeatable read	Phantom read	Lost update	Write skew
`READ UNCOMMITTED`	*Same as `READ COMMITTED` in PostgreSQL* – it never reads uncommitted data				
`READ COMMITTED` (default)	Prevented	Possible	Possible	Possible	Possible
`REPEATABLE READ`	Prevented	Prevented	Prevented in PostgreSQL	Prevented	Possible
`SERIALIZABLE`	Prevented	Prevented	Prevented	Prevented	Prevented

- **Real-time – the desk:** one row, the check inside the `UPDATE` – the default is enough.
- **No overwrite – the overdue clean-up:** list and count agree, no lost mark over a return – `REPEATABLE READ`.
- **Valid decision – the loan limit:** `SERIALIZABLE` in every transaction that applies the rule – and retry.

CHECK AND WRITE IN ONE STEP

```
BEGIN;
UPDATE Copy SET status = 'on loan'
WHERE book_id = 5 AND copy_number = 1 AND status = 'available';
-- the program checks: did this change exactly 1 row?
```

Read it as: *take the copy – but only if it is still available.*

- **1 row changed:** this desk won. Go on: `INSERT INTO Loan ...; COMMIT;`
- **0 rows changed:** someone else already took it. `ROLLBACK;` and tell the member – no double booking.
- **Why not `SELECT` first, then `UPDATE`?** Between the two statements there is a gap, and another desk can slip into it. Inside one `UPDATE`, there is no gap.

TWO TERMINALS, ONE COPY

```
-- Terminal A
BEGIN;
UPDATE Copy SET status = 'on loan'
WHERE book_id = 1 AND copy_number = 1
  AND status = 'available';




COMMIT;
```

```
-- Terminal B




UPDATE Copy SET status = 'on loan'
WHERE book_id = 1 AND copy_number = 1
  AND status = 'available';
-- <- what happens here?
```

Read it as: *A has written the row but not committed. B tries to write the same row.*

- **Guess first:** does B (a) succeed and change 1 row, (b) succeed and change 0 rows, or (c) something else?

WHAT ACTUALLY HAPPENED

```
-- Terminal A
BEGIN;
UPDATE Copy SET status = 'on loan'
WHERE ...;
-- written, NOT committed yet




COMMIT;
```

```
-- Terminal B




UPDATE Copy SET status = 'on loan'
WHERE ...;
-- does not return: WAITING (~2.5 s)

-- returns at once: UPDATE 0
```

Read it as: *B waited for A, then checked its `WHERE` again – and found the copy already gone.*

- **B neither failed nor double-booked.** It waited, then honestly reported `UPDATE 0`.
- **The row lock:** a written row stays locked until `COMMIT` or `ROLLBACK`; only a second *writer* of the same row waits. Readers never wait for writers.

DEADLOCK: WAITING FOR EACH OTHER

```
-- Terminal A
BEGIN; UPDATE Copy ...
  WHERE book_id = 1 AND copy_number = 1;

UPDATE Copy ...   -- waits for B
  WHERE book_id = 3 AND copy_number = 1;
```

```
-- Terminal B

BEGIN; UPDATE Copy ...
  WHERE book_id = 3 AND copy_number = 1;

UPDATE Copy ...   -- waits for A
  WHERE book_id = 1 AND copy_number = 1;
```

```
ERROR:  deadlock detected
DETAIL:  Process 144 waits for ShareLock on transaction 761; blocked by process 151.
         Process 151 waits for ShareLock on transaction 760; blocked by process 144.
```

Read it as: *Anna's desk holds copy (1, 1), Ben's holds (3, 1) – each needs the other's; neither can go on.*

- **After about a second, PostgreSQL cancels one** and undoes its work. The other goes on and commits.
- **The cancelled one must be retried** – by your program, not by the database.

AVOIDING DEADLOCKS

Read it as: *you cannot stop the database from cancelling – but you can stop the cycle from forming.*

- **Lock in the same order everywhere.** If both desks always touch the lower copy first, the second desk just waits – no cycle can form.
- **Keep transactions short.** The less time a lock is held, the smaller the chance of a cycle.
- **Be ready to retry.** Catch `deadlock detected` and run the whole transaction again.
- **Behind it: two-phase locking.** A transaction takes locks as it goes and releases them all at `COMMIT`. That is what makes it safe – and why two growing transactions can block each other.

INDEX: NEED 2

Function	What the database provides	Read more
F2.1.1 Latest commits	no dirty reads `READ COMMITTED`, the default	DBS 17.8
F2.1.2 Snapshot + write protection	no non-repeatable or phantom reads; no lost update `REPEATABLE READ`	DBS 17.8, 18.8
F2.1.3 Snapshot + basis protection	serializable; write skew; retry when refused `SERIALIZABLE`	DBS 17.8–17.9
F2.2 Row waits	row lock; the check inside the write the second `UPDATE` waits, then `UPDATE 0`	DBS 18.1
F2.3 Deadlock detection	deadlock; lock order; retry lock the lower copy first	DBS 18.2

DBS = *Database System Concepts*, Silberschatz, Korth & Sudarshan, 7th ed. (17.1 = chapter 17, section 1); PG = the PostgreSQL documentation, page for that command.

BREAK

5-MINUTE BREAK

Stretch, grab water. Next: the four promises behind everything you just saw have a name – ACID.

ACID: WHAT A TRANSACTION PROMISES

The key definition of this lecture: four promises every relational database makes

ACID: THE DEFINITION

A *transaction* is a unit of work. A relational database promises four things about every transaction:

	Promise	Definition
A	**Atomicity**	All of the transaction's changes happen, or none of them do. There is never a "half done" state – not even after a crash.
C	**Consistency**	A transaction takes the database from one valid state to another: every declared rule (`PRIMARY KEY`, `FOREIGN KEY`, `CHECK`, `NOT NULL`, `UNIQUE`) still holds after `COMMIT`.
I	**Isolation**	Transactions running at the same time do not see each other's unfinished work – each runs as if it were alone. *How* strictly is chosen by the isolation level.
D	**Durability**	Once `COMMIT` returns, the change survives – a crash a moment later does not undo it.

Stable, accurate data in complex situations is exactly these four promises.

IN THE LIBRARY: THE AUDITOR'S FOUR QUESTIONS

The city's auditor checks the desk program. One question per letter – each about one loan:

	The auditor asks	What could go wrong	Why it cannot
A	Can a loan be half recorded?	The power fails between the two writes: copy (5, 1) is *on loan*, but no loan says who has it.	Both writes are kept, or neither.
C	Can a loan break the library's rules?	A loan for member 999, who does not exist.	The `FOREIGN KEY` refuses it; it is never committed.
I	Can a desk act on another desk's unfinished loan?	Desk B lends copy (1, 1) while desk A is still lending it – two receipts.	B waits for A's `COMMIT`, then gets `UPDATE 0`.
D	Once the desk says "done", can the loan vanish?	The receipt is printed, then the server crashes.	"Done" means `COMMIT` returned: the loan is already on disk.

Read it as: *the auditor asks "can I trust this record?" – ACID is the database's four-part answer.*

ACID IN TODAY'S LECTURE

	What it rules out, concretely	Where you met it
A	Copy (5, 1) marked *on loan* with no loan that says who has it	**L7.F1.1** All or nothing: `BEGIN … COMMIT`
C	A loan that breaks a `CHECK` or a `FOREIGN KEY` ever being committed	**L3** Constraints declared: `PRIMARY KEY`, `FOREIGN KEY`, `CHECK` **L6.N4** Something leaves that others depend on
I	Two desks' borrows corrupting each other; a clean-up reading half-done work	**L7.N2** Many people work at once: locks, isolation levels
D	A crash right after `COMMIT` undoing a confirmed loan	**L7.F1.3** Stable once committed: the write-ahead log

L7 = Lecture 7; N2 = Need 2 of that lecture; F1.1 = what the database offers for that need (F1.x belongs to Need 1, F2.x to Need 2, …).

- **None of the four is free.** Stronger isolation means waiting and retries; durability means writing to disk.
- **None of them decides what belongs in one transaction.** Which statements go between `BEGIN` and `COMMIT` is your call – put both writes of one loan in one transaction, or atomicity cannot help.

NEED 3: LARGE DATA, STILL FAST

The loan log reaches 500,000 rows, and “everything member 27 ever borrowed” gets slow.

SQL: `CREATE INDEX`, `EXPLAIN`

NEED 3: LARGE DATA, STILL FAST

More tables, more joins, more rows – and some questions get slow. The database helps in two ways; one choice is yours.

WHAT THE DATABASE CAN PROVIDE FOR YOU

F3.1 Find rows without reading them all

An index jumps straight to the matching rows.

F3.2 See how a query is run

The plan shows whether the whole table is read – and how long it took.

WHAT YOU MUST DECIDE

Which columns deserve an index?

An index speeds up reads but slows down every write.

Index a column only if both hold:

1. You often search or join on it, e.g. `member_id`.
2. A search on it finds only a few rows: 10,000 of 500,000.

Not every column "to be safe": writes get slower, reads no faster.

IN THE LIBRARY: THE LOG REACHES 500,000 ROWS

The loan log has grown to 500,000 rows – three questions at the desk.

FIND IT FAST

Member 27's loans

The desk asks for everything member 27 ever borrowed – 10,000 loans. The answer used to come at once; now it takes noticeably long.

SEE WHAT HAPPENS

Why is it slow?

Before changing anything, find out: does the database read all 500,000 rows to find member 27's 10,000 – or does it jump straight to them?

YOUR DECISION

Index every column?

The new librarian asks for an index on every column of the log, "to be safe". Then every loan and every return must update all of those indexes too.

WHAT THE DATABASE OFFERS FOR NEED 3

	Function	SQL	What it does
F3.1	Find rows without reading them all	`CREATE INDEX` (a B-tree)	Jumps to the matching rows instead of reading the whole table
F3.2	See how a query is run	`EXPLAIN`, `EXPLAIN ANALYZE`	Shows the plan – a full scan or an index. `ANALYZE` also runs it and times it

What it cannot do for you: choose which columns to index. Every index makes every write a little slower.

WHAT IS AN INDEX?

```
CREATE INDEX idx_loanlog_member_id ON LoanLog(member_id);
DROP INDEX idx_loanlog_member_id;
```

Read it as: *keep a sorted list of `member_id` values, each pointing at its rows.*

- **Like the index at the back of a book.** Look the word up once, jump to the right pages – instead of reading every page. Without it, `WHERE member_id = 27` must read every row: a *Seq Scan*.
- **A B-tree, kept sorted.** It serves `=`, `<`, `>`, `BETWEEN`, `ORDER BY` – and joins on that column. Lookups grow only slowly as the table grows.
- **A separate object.** It never changes *what* a query returns – only *how fast*. And it is updated on every write.

AN INDEX, DRAWN

THE TABLE – ROWS IN THE ORDER THEY WERE WRITTEN

row	member_id	book
1	5	Dune
2	**27**	Emma
3	8	Ulysses
4	**27**	Hamlet
5	31	Dune
6	5	Emma
…	…	…
1210	**27**	Ulysses
…	…	… 500,000 rows

THE INDEX – A B-TREE, SIMPLIFIED

ROOT PAGE – WHICH LEAF PAGE TO READ

member_id	go to
1 – 11	leaf page 1
12 – 30	leaf page 2
31 – …	leaf page 3, …



LEAF PAGE 2 – SORTED, POINTING AT THE ROWS

member_id	where its rows are
12	rows 7, …
27	rows 2, 4, 1210, … (10,000)
30	rows …

Find 27: read the root, jump to leaf page 2, fetch the rows – a few page reads, never the whole list.

- **Without the index:** read rows 1 to 500,000 and check each one – a *Seq Scan*.
- **With the index:** root page → leaf page 2 → rows 2, 4, 1210, … directly – an *Index Scan*.

CLUSTERED VS. NON-CLUSTERED

```
-- Non-clustered: the index sits beside the table (every index in PostgreSQL)
CREATE INDEX idx_loanlog_member_id ON LoanLog(member_id);
-- Clustered, once: rewrite the table in the order of that index (member_id)
CLUSTER LoanLog USING idx_loanlog_member_id;
```

Read it as: *does the index sit beside the table, or is the table itself sorted by it?*

- **Non-clustered:** a separate structure with the values and a pointer to each row. Every PostgreSQL index is this kind – including the one a `PRIMARY KEY` creates.
- **Clustered:** the rows themselves are stored in index order – at most one per table. SQL Server and MySQL do this for the primary key.
- **PostgreSQL's `CLUSTER`** sorts a table by an index *once*. New rows afterwards go wherever there is space.

CLUSTERED, DRAWN

BEFORE CLUSTER – ROWS WHERE THEY WERE WRITTEN

page	member_id	book
1	5	Dune
1	**27**	Emma
1	8	Ulysses
2	**27**	Hamlet
2	31	Dune
2	5	Emma
…	…	…
404	**27**	Ulysses
…	…	…

AFTER CLUSTER – ROWS IN MEMBER_ID ORDER

page	member_id	book
1	5	Dune
1	5	Emma
1	8	Ulysses
…	…	…
k	**27**	Emma
k	**27**	Hamlet
k	**27**	Ulysses
k+1	31	Dune
…	…	…

- **Before:** the index finds member 27′s rows, but they sit on many different pages – each may cost a page read.
- **After:** they sit together on a few neighbouring pages – far fewer pages to read.
- **But only once:** a loan added tomorrow goes wherever there is space – run CLUSTER again to restore the order.

READING A PLAN: EXPLAIN

```
EXPLAIN SELECT * FROM LoanLog WHERE member_id = 27;
```

```
Seq Scan on loanlog  (cost=0.00..9927.00 rows=9750 width=31)
  Filter: (member_id = 27)
```

Read it as: *“I will read the whole table and keep the rows for member 27.”*

- **EXPLAIN only plans.** The `cost` numbers are estimates in arbitrary units – good for comparing plans, not for predicting milliseconds.
- **EXPLAIN ANALYZE really runs the query** and adds real times and row counts. On an `UPDATE` it really writes – so wrap it: `BEGIN; EXPLAIN ANALYZE UPDATE ...; ROLLBACK;`

AN INDEX THAT HELPS

```
-- Before:  EXPLAIN ANALYZE SELECT * FROM LoanLog WHERE member_id = 27;
Seq Scan on loanlog  (actual time=0.005..10.325 rows=10045 loops=1)
  Filter: (member_id = 27)
  Rows Removed by Filter: 489955
Execution Time: 10.521 ms

-- After:   CREATE INDEX idx_loanlog_member_id ON LoanLog(member_id);
Bitmap Heap Scan on loanlog  (actual time=0.578..2.664 rows=10045 loops=1)
  ->  Bitmap Index Scan on idx_loanlog_member_id
Execution Time: 2.843 ms
```

Read it as: *before, 489,955 rows were read and thrown away; after, the index leads straight to the 10,045 that match.*

- **About 3.7 times faster** in this run. Your milliseconds will differ – the plan will not.
- **Why it helps:** member 27 is about 2% of the rows. An index pays off when few rows match.

WHEN DOES AN INDEX PAY OFF?

1 · KEY POINT

NEED 3 DECISION ↑

Ask three questions about the column – the answers decide.

Question	An index helps when …	An index does not help when …
How many rows match?	**few** – `member_id`: 2% each	**many** – `event_type`: half the table
How big is the table?	**large**	**small** – reading it all is already fast
How is the column used?	**searched often** – in `WHERE`, `JOIN`, `ORDER BY`	**written a lot** – every write also updates the index

Verified – an index on `event_type`: all cost, no gain.

- **The column:** it says what happened – only two values, `'borrow'` or `'return'`, each about half of the 500,000 rows.
- **Writes got slower:** the same 200,000-row `INSERT` took 827 ms without it, 1,065 ms with it – about 1.3 times slower.
- **Reads got no faster:** counting the `'borrow'` rows still read half the table.

INDEX: NEED 3

Function	What the database provides	Read more
F3.1 Find rows without reading them all	B-tree index; few matching rows `CREATE INDEX idx_loanlog_member_id ON LoanLog(member_id);`	DBS 14.1–14.3
F3.2 See how a query is run	the query plan `EXPLAIN ANALYZE SELECT ...;` — `Seq Scan` or an index scan	PG `EXPLAIN`
	`EXPLAIN` VS `EXPLAIN ANALYZE` `BEGIN; EXPLAIN ANALYZE UPDATE ...; ROLLBACK;`	PG “Using EXPLAIN”
Decision: Which columns?	write cost; many matching rows don’t: an index on a 2-value column made inserts slower and reads no faster	DBS 14.2

DBS = *Database System Concepts*, Silberschatz, Korth & Sudarshan, 7th ed. (17.1 = chapter 17, section 1); PG = the PostgreSQL documentation, page for that command.

WRAP-UP

LECTURE 7 IN NEED ASPECT: WHAT THE WORLD NEEDS

	Need	What the database can provide	What you must decide
1	A complex operation, done safely	**F1.1** All or nothing **F1.2** Go back to a save point **F1.3** Stable once committed	Design one complete operation as one transaction
2	Many people work at once	**F2.1.1** Latest commits **F2.1.2** Snapshot + write protection **F2.1.3** Snapshot + basis protection **F2.2** Row waits **F2.3** Deadlock detection	Choose the right isolation level for each situation; the check inside the `UPDATE`; lock order; retry
3	Large data, still fast	**F3.1** Find rows fast **F3.2** See the plan	Which columns deserve an index?

Needs 1 and 2 keep the data **stable and accurate** – the ACID promises. Need 3 keeps it **efficient**.

LECTURE 7 IN PRACTICE ASPECT: WHAT THE DATABASE OFFERS

Each need, the functions that serve it – and the SQL behind each one.

Need	Function	What it does	In SQL
1 A complex operation, done safely	**F1.1**	All or nothing	`BEGIN ... COMMIT` / `ROLLBACK`
	F1.2	Go back to a save point	`SAVEPOINT`, `ROLLBACK TO SAVEPOINT`
	F1.3	Stable once committed	`COMMIT` (write-ahead log)
2 Many people work at once	**F2.1.1**	Latest commits	`READ COMMITTED` – the default
	F2.1.2	Snapshot + write protection	`REPEATABLE READ`
	F2.1.3	Snapshot + basis protection	`SERIALIZABLE`
	F2.2	Row waits	row locks – automatic
	F2.3	Deadlock detection	automatic – one is cancelled
3 Large data, still fast	**F3.1**	Find rows without reading them all	`CREATE INDEX`
	F3.2	See how a query is run	`EXPLAIN`, `EXPLAIN ANALYZE`

The SQL is the part that changes with the tool. The needs, the ACID promises and the decisions stay.

WHERE TO READ MORE

Concept	Where
`BEGIN`, `COMMIT`, `ROLLBACK` in SQL	DBS ch. 4.3 *Transactions*
Transactions, ACID	DBS ch. 17.1–17.4 *Transactions* (concept level)
Isolation levels, anomalies	DBS ch. 17.8–17.9, and 18.8 *Snapshot Isolation* (how PostgreSQL does `REPEATABLE READ`); PostgreSQL docs "Transaction Isolation" (short, essential)
Locks, deadlock, two-phase locking	DBS ch. 18.1–18.2 (overview) – the rest is *Database Systems*, 6th semester
Durability, the write-ahead log	DBS ch. 19.3 and 19.5 (the log and the write-ahead rule); PostgreSQL docs "Reliability and the Write-Ahead Log"
Indexes, B-trees	DBS ch. 14.1–14.3 *Indexing* (14.3 = B+-trees), 4.6 *Index Definition in SQL*; PostgreSQL docs `CREATE INDEX`
Query plans	PostgreSQL docs "Using EXPLAIN"

DBS = *Database System Concepts*, Silberschatz, Korth & Sudarshan, 7th ed. (17.1 = chapter 17, section 1); PostgreSQL docs = the official PostgreSQL documentation, page for that topic.

EXERCISES

	Need	Ex.	DB function	What you do – exercise sheet 7
1	A complex operation, done safely	1	F1	Borrow a copy in one transaction; try, check and go back with `SAVEPOINT`
2	Many people work at once	2, 4	F1, F2.2, F2.3	Sell one seat twice – then not, with a transaction and the check; cause a deadlock and fix it
	ACID and isolation	5	F2.1	Name the ACID letter and the anomaly; choose the weakest safe level
3	Large data, still fast	3, 6	F3	Index `LoanLog` before and after; decide which small-table indexes would help
	Your own project	7	F1, F2.1, F3	Three transactions that must be atomic; indexes on two columns, with `EXPLAIN ANALYZE`

Every statement comes with three lines: **1.** which Index row it belongs to **2.** why it is right for your schema **3.** how you checked it – a second terminal, a row count, a plan.

SUMMARY

Lecture 7 – keeping data stable, accurate and efficient when things get complex:

1. **A complex operation, done safely.** `BEGIN ... COMMIT` or `ROLLBACK` makes several steps one; `SAVEPOINT` lets you try, check and go back; after `COMMIT`, the change is stable.
2. **Many people work at once.** Choose an isolation level per transaction – real-time, no overwrite or a valid decision; row locks make the second writer wait; lock in one order and retry after a deadlock.
3. **Large data, still fast.** Index the columns that pick out few rows; check with `EXPLAIN ANALYZE`; do not index everything.

ACID names the promises behind 1 and 2: atomicity, consistency, isolation, durability.

Next lecture – the front desk gets its own program: everything you did in SQL today, now from Java with JDBC.